

TASK1 量化交易初体验：从零搭建数据引擎

在量化交易领域，数据是构建一切交易策略的基石。量化交易凭借其系统性、纪律性和可重复性，逐渐成为金融领域的热门方向。而搭建一个稳定、高效的量化数据引擎是开展量化交易的首要一步，它不仅为后续的策略开发、回测和实盘交易提供基础数据支持，更是理解量化交易流程的关键环节。

任务 1. 相较于传统手工操作交易的方法，量化交易有哪些优势？

我认为量化交易的主要优势可以用 2 点来概括，一是将策略转化为一套自动执行的规则，从而减少人主观看法和手工操作带来的不确定性，二是比人有更强的计算性能，从而能够处理大规模数据，同时大规模地进行回测。

任务 2. 了解以下基本概念并解释：K 线，基本面，技术面

K 线是最常见的价格展示方式，用来反映某一时间周期内价格的变化情况。一根 K 线主要由**开盘价**、**收盘价**、**最高价**和**最低价**四个价格构成。其中，K 线中间较粗的部分称为**实体**，表示开盘价和收盘价之间的价格区间。实体上方延伸出来的细线称为**上影线**，反映价格曾经上涨到更高的位置但最终未能维持。实体下方延伸出来的细线称为**下影线**，说明价格曾一度跌到更低的位置后又有所回升。

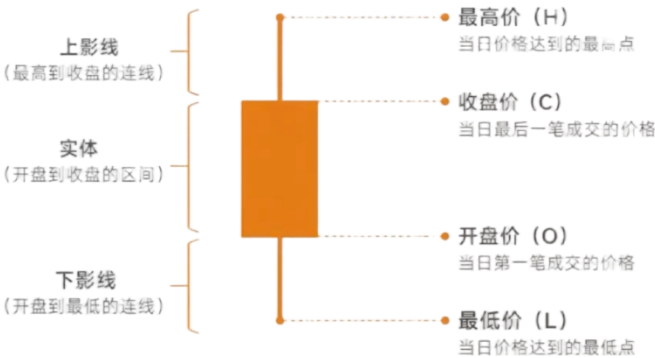


图 1 K 线

基本面是指影响资产内在价值的各种因素。对于股票来说，基本面分析主要关注一家公司的内部经营状况，例如营业收入、利润、现金流、资产负债情况、行业前景以及管理层能力等，同时也会结合宏观经济环境、利率、政策等外部因素进行综合分析。基本面分析的**目的**是判断一项资产是否具有投资价值，即估计它的真实价值是否高于或低于当前市场价格。如果一家公司的经营持续向好、盈利能力不断提升，那么从基本面的角度来看，其股价长期上涨的可能性通常也会更大。

技术面是指通过研究市场历史交易数据来分析未来价格走势的方法, 其研究对象主要包括价格、成交量以及各种技术指标, 而不是公司的经营情况。技术分析认为, 市场上的信息最终都会反映到价格之中, 因此可以通过观察 K 线形态、趋势、成交量变化以及移动平均线等指标, 寻找价格运行的规律, 从而辅助判断买卖时机。与基本面分析相比较, 技术面的**目的**主要在于关注市场参与者的行为和资金流向, 更侧重回答买卖时机的问题。在实际投资中, 基本面与技术面可以被结合使用, 既选择具有长期价值的标的, 又利用技术分析寻找更合适的交易时机。

任务 3. 注册 <https://www.tushare.pro/> 平台, 获得查询 key, python 编程实现以下内容

1) 获取沪深股市中某一股票过去一年内每个交易日交易数据

Py 代码

"""

fetch_data.py

=====

使用 TuShare 获取沪深股市股票的日线交易数据, 并保存为 CSV。

用法:

```
python src/fetch_data.py
```

前置条件:

1. 注册 TuShare Pro 账号: <https://tushare.pro>
2. 获取 token: 登录后进入「个人中心」→「接口 TOKEN」
3. 设置环境变量: `export TUSHARE_TOKEN="your_token_here"`

本脚本默认获取:

- 股票: 688256.SH (寒武纪, 芯片, 科创板)
- 时间: 过去一年 (2025-07-01 至 2026-07-01)

"""

```
import os
```

```
import sys
```

```
import pandas as pd
```

```
import tushare as ts
```

```
# 股票代码 (TuShare 格式: 代码 + 交易所后缀)
```

```
# 上海证券交易所: .SH | 深圳证券交易所: .SZ
```

```

# 科创板 688xxx
TS_CODE = "688256.SH" # 寒武纪 - 芯片
START_DATE = "20250701" # 起始日期 YYYYMMDD
END_DATE = "20260701" # 结束日期 YYYYMMDD
# 输出路径
OUTPUT_DIR = os.path.join(os.path.dirname(__file__), "..", "data",
"raw")
OUTPUT_FILE = os.path.join(OUTPUT_DIR, "688256_daily.csv")

def get_token() -> str:
    """从环境变量获取 TuShare token。"""
    token = os.environ.get("TUSHARE_TOKEN", "")
    return token

def fetch_daily(ts_code: str, start_date: str, end_date: str)
-> pd.DataFrame:
    """
    通过 TuShare Pro API 获取 A 股日线行情。

    参数
    ----
    ts_code : str
        TuShare 股票代码，如 '688256.SH'
    start_date : str 起始日期，格式 YYYYMMDD
    end_date : str 结束日期，格式 YYYYMMDD

    返回
    ----
    pd.DataFrame，包含以下字段：
    ts_code - 股票代码
    trade_date - 交易日期
    open - 开盘价（元）
    high - 最高价（元）
    low - 最低价（元）
    close - 收盘价（元）
    pre_close - 前收盘价（元）
    change - 涨跌额（元）
    pct_chg - 涨跌幅（%）

```

```

vol - 成交量 (手)
amount - 成交额 (千元)
"""

token = get_token()
pro = ts.pro_api(token)
df = pro.daily(
    ts_code=ts_code,
    start_date=start_date,
    end_date=end_date,
)
# 按交易日期升序排列
df = df.sort_values("trade_date").reset_index(drop=True)
return df

def save_to_csv(df: pd.DataFrame, filepath: str) -> None:
    """将 DataFrame 保存为 CSV 文件。"""
    os.makedirs(os.path.dirname(filepath), exist_ok=True)
    df.to_csv(filepath, index=False, encoding="utf-8-sig")
    print(f"    数据已保存至: {filepath}")

def main() -> None:
    df = fetch_daily(TS_CODE, START_DATE, END_DATE)
    save_to_csv(df, OUTPUT_FILE)

if __name__ == "__main__":
    main()

```

运行结果

	A	B	C	D	E	F	G	H	I	J	K
1	ts_code	trade_date	open	high	low	close	pre_close	change	pct_chg	vol	amount
2	688256.SH	20250701	598.88	598.88	560.66	563	601.5	-38.5	-6.4007	104237.9	5924880.8
3	688256.SH	20250702	560.22	560.22	541.91	545.1	563	-17.9	-3.1794	64098.04	3509933.37
4	688256.SH	20250703	546	553.6	544	547.3	545.1	2.2	0.4036	39722.89	2173734.83
5	688256.SH	20250704	545	557.97	538.67	547.47	547.3	0.17	0.0311	56859.33	3111587.18
6	688256.SH	20250707	544	553.86	541.01	541.38	547.47	-6.09	-1.1124	33310.55	1810291.92
7	688256.SH	20250708	540	545.3	538.1	542.77	541.38	1.39	0.2568	39527.81	2143484.03
8	688256.SH	20250709	544	544.78	533.5	535	542.77	-7.77	-1.4315	49069.59	2643083.78
9	688256.SH	20250710	534.01	537.87	520.67	523.5	535	-11.5	-2.1495	85815.16	4522151.66
10	688256.SH	20250711	523	559.96	523	553.98	523.5	30.48	5.8223	116436.72	6359225.56
11	688256.SH	20250714	555	570	554	565.06	553.98	11.08	2.0001	65114.93	3678055.66
12	688256.SH	20250715	564	564	546.76	553.1	565.06	11.06	2.1166	50695.90	3200306.92

图2 寒武纪过去一年交易日交易数据

2) 画出每日收盘价的曲线图

Py 代码

"""

visualize.py

=====

读取股票日线数据 CSV，绘制每日收盘价曲线图

用法：

```
python src/visualize.py [csv_path]
```

- 不传参数：默认读取 data/raw/688256_daily.csv

输出：

收盘价曲线图 → output/figures/688256_close_price.png

"""

```
import os
import sys
import matplotlib.pyplot as plt
import matplotlib.dates as mdates
import pandas as pd
plt.rcParams["font.sans-serif"] = ["Arial Unicode MS", "SimHei",
    "Heiti TC", "sans-serif"]
plt.rcParams["axes.unicode_minus"] = False
DEFAULT_CSV = os.path.join(os.path.dirname(__file__), "..", "data",
    "raw", "688256_daily.csv")
FIGURE_OUTPUT = os.path.join(os.path.dirname(__file__), "..",
    "output", "figures", "688256_close_price.png")

def load_data(csv_path: str) -> pd.DataFrame:
    """读取 CSV 并做基础处理。"""
    df = pd.read_csv(csv_path, encoding="utf-8-sig")
    required_cols = {"trade_date", "close"}
    df["trade_date"] = pd.to_datetime(df["trade_date"],
format="%Y%m%d")
    df = df.sort_values("trade_date").reset_index(drop=True)
    return df

def plot_close_price(df: pd.DataFrame, ts_code: str, output_path:
```

```

str) -> None:
    """绘制每日收盘价曲线图并保存。"""
    fig, ax = plt.subplots(figsize=(14, 6))
    ax.plot(
        df["trade_date"], df["close"], linewidth=1.2,
        color="#1f77b4", marker="", alpha=0.9
    )

    # 标注最高/最低收盘价
    max_idx = df["close"].idxmax()
    min_idx = df["close"].idxmin()
    ax.annotate(
        f"最高: {df['close'][max_idx]:.2f}",
        xy=(df["trade_date"][max_idx], df["close"][max_idx]),
        xytext=(0, 12), textcoords="offset points",
        fontsize=9, color="red", ha="center",
        arrowprops=dict(arrowstyle="->", color="red", lw=0.8),
    )
    ax.annotate(
        f"最低: {df['close'][min_idx]:.2f}",
        xy=(df["trade_date"][min_idx], df["close"][min_idx]),
        xytext=(0, -16), textcoords="offset points",
        fontsize=9, color="green", ha="center",
        arrowprops=dict(arrowstyle="->", color="green", lw=0.8),
    )

    ax.xaxis.set_major_formatter(mdates.DateFormatter("%Y-%m"))
    ax.xaxis.set_major_locator(mdates.MonthLocator(interval=1))
    plt.setp(
        ax.get_xticklabels(), rotation=45, ha="right", fontsize=8
    )

    # 标签与标题
    ax.set_xlabel("Date", fontsize=11)
    ax.set_ylabel("Closing Price (CNY)", fontsize=11)
    ax.set_title(
        f"{ts_code} - Daily Closing Price
        ({df['trade_date'].min().strftime('%Y-%m-%d')} to

```

```

        {df['trade_date'].max().strftime('%Y-%m-%d')})"),
        fontsize=13, fontweight="bold"
    )
    ax.grid(True, linestyle="--", alpha=0.4)
    fig.tight_layout()

    os.makedirs(os.path.dirname(output_path), exist_ok=True)
    fig.savefig(output_path, dpi=150, bbox_inches="tight")
    plt.close(fig)

def main() -> None:
    # 解析命令行参数
    csv_path = sys.argv[1] if len(sys.argv) > 1 else DEFAULT_CSV
    df = load_data(csv_path)
    # 从 CSV 中提取股票代码（如果有 ts_code 列），否则从文件名推断
    if "ts_code" in df.columns:
        ts_code = df["ts_code"].iloc[0]
    else:
        ts_code = os.path.splitext(os.path.basename(csv_path))[0]
    plot_close_price(df, ts_code, FIGURE_OUTPUT)

if __name__ == "__main__":
    main()

```

运行结果



图3 寒武纪过去一年交易日收盘价

3) 将数据保存成 csv 格式数据, 以备后续任务使用

Py 代码

"""

visualize.py

=====

读取股票日线数据 CSV, 导出处理后的数据。

用法:

```
python src/visualize.py [csv_path]
```

- 不传参数: 默认读取 data/raw/688256_daily.csv

输出:

处理后的 CSV → data/processed/688256_daily_processed.csv

"""

```
import os
```

```
import sys
```

```
import matplotlib.pyplot as plt
```

```
import matplotlib.dates as mdates
```

```
import pandas as pd
```

```
DEFAULT_CSV = os.path.join(
    os.path.dirname(__file__), "..", "data", "raw",
    "688256_daily.csv"
)
```

```
PROCESSED_OUTPUT = os.path.join(
    os.path.dirname(__file__), "..", "data", "processed",
    "688256_daily_processed.csv"
)
```

```
def load_data(csv_path: str) -> pd.DataFrame:
    """读取 CSV 并做基础处理。"""
    df = pd.read_csv(csv_path, encoding="utf-8-sig")
    required_cols = {"trade_date", "close"}
    missing = required_cols - set(df.columns)
    if missing:
        print(f"CSV 缺少必要列: {missing}")
        sys.exit(1)
```



```
# 转换日期列
df["trade_date"] = pd.to_datetime(df["trade_date"],
format="%Y%m%d")
df = df.sort_values("trade_date").reset_index(drop=True)
return df

def save_processed_csv(df: pd.DataFrame, output_path: str) -> None:
    """保存处理后的数据为 CSV"""
    os.makedirs(os.path.dirname(output_path), exist_ok=True)
    # 保留原始日期格式便于后续使用
    df_out = df.copy()
    df_out["trade_date"] =
df_out["trade_date"].dt.strftime("%Y%m%d")
    df_out.to_csv(output_path, index=False, encoding="utf-8-sig")

def main() -> None:
    csv_path = sys.argv[1] if len(sys.argv) > 1 else DEFAULT_CSV
    df = load_data(csv_path)
    save_processed_csv(df, PROCESSED_OUTPUT)

if __name__ == "__main__":
    main()
```

运行结果

	A	B	C	D	E	F	G	H	I	J	K
1	ts_code	trade_date	open	high	low	close	pre_close	change	pct_chg	vol	amount
2	688256.SH	20250701	598.88	598.88	560.66	563	601.5	-38.5	-6.4007	104237.9	5924880.8
3	688256.SH	20250702	560.22	560.22	541.91	545.1	563	-17.9	-3.1794	64098.04	3509933.37
4	688256.SH	20250703	546	553.6	544	547.3	545.1	2.2	0.4036	39722.89	2173734.83
5	688256.SH	20250704	545	557.97	538.67	547.47	547.3	0.17	0.0311	56859.33	3111587.18
6	688256.SH	20250707	544	553.86	541.01	541.38	547.47	-6.09	-1.1124	33310.55	1810291.92
7	688256.SH	20250708	540	545.3	538.1	542.77	541.38	1.39	0.2568	39527.81	2143484.03
8	688256.SH	20250709	544	544.78	533.5	535	542.77	-7.77	-1.4315	49069.59	2643083.78
9	688256.SH	20250710	534.01	537.87	520.67	523.5	535	-11.5	-2.1495	85815.16	4522151.66
10	688256.SH	20250711	523	559.96	523	553.98	523.5	30.48	5.8223	116436.72	6359225.56
11	688256.SH	20250714	555	570	554	565.06	553.98	11.08	2.0001	65114.93	3678055.66
12	688256.SH	20250715	564	564	546.76	553.1	565.06	11.06	2.1166	50695.90	3200306.92

图 4 寒武纪过去一年交易日交易数据

4) 配置 MCP

在 .claude 中配置 settings.local.json 文件，这样就可以直接使用自然语言获取数据了。

```
"mcpServers": {  
  "tushareMcp": {  
    "type": "url",  
    "url": "https://api.tushare.pro/mcp/token=我的 api"  
  }  
}
```